

A Survey of Four Notable Approaches to Neural Signed Distance Functions

Companion to the main report; deep-dive into the four most influential papers in the line

Author: Ihor Ivanyshyn **Date:** 17 May 2026

Introduction

The task — fitting a neural SDF to a 3D mesh under tight memory and latency budgets — has been approached from many angles since the field opened in 2019. This document presents four of the most influential and conceptually diverse methods, chosen to span the design space and to trace the trajectory the field followed:

1. **DeepSDF** — a *purely implicit* approach: all the geometry lives inside a large MLP, with no spatial data structure at all.
2. **SIREN** — a *purely implicit, frequency-aware* approach: still all MLP, but the activations are chosen to natively represent high-frequency signals.
3. **NGLOD** — a *hybrid explicit/implicit* approach: most of the geometry is stored in an octree of feature vectors, with a small MLP as the decoder.
4. **Instant-NGP** — the *hybrid with constant-time access*: replaces the octree with multi-resolution hash tables, keeps the small MLP. This is the method our implementation builds on.

The progression from (1) to (4) is precisely the trajectory the field followed: pure MLPs proved expressive but slow and over-parameterized; spatial data structures absorbed most of the parameter count, leaving the MLP as a final projection. The transition from (3) to (4) is the move from explicit spatial indexing (octree traversal) to implicit indexing (hashing), which trades tree-construction overhead for hash collisions — a trade that turns out to be overwhelmingly favorable at the resolutions of interest.

1. DeepSDF (Park et al., CVPR 2019)

1.1 Problem and motivation

Before 2019, 3D shape representations were either discrete (voxel grids, point clouds, meshes) or based on implicit functions defined explicitly (signed distance from a set of geometric primitives). Discrete representations have memory issues at resolution; explicit implicit functions cannot represent arbitrary smooth geometry.

DeepSDF [1] proposed the now-standard formulation: represent a shape as a learned function

$$f_{\theta} : \mathbb{R}^3 \rightarrow \mathbb{R}, \quad f_{\theta}(\mathbf{x}) = \text{SDF}(\mathbf{x})$$

where θ are the parameters of a feed-forward neural network. The network's *weights* implicitly encode the shape, and the surface is recovered as the zero level set $\{ \mathbf{x} : f_{\theta}(\mathbf{x}) = 0 \}$.

This was a radical departure: the entire 3D object becomes a function lookup, with no spatial primitives at all.

1.2 Architecture

The proposed network is an 8-layer feed-forward MLP with 512 hidden units per layer, ReLU activations, and a skip connection from the input to the fourth layer (mirroring the U-Net idea in 1D):

```
input (x, y, z)
  |
  ▼
Linear(3 → 512) → ReLU
Linear(512 → 512) → ReLU
Linear(512 → 512) → ReLU
Linear(512 → 509) → ReLU
  |
  ▼ concat with input (skip)
Linear(509 + 3 → 512) → ReLU
Linear(512 → 512) → ReLU
Linear(512 → 512) → ReLU
Linear(512 → 1) → tanh
  |
  ▼
SDF (clipped to [-6, 6])
```

The tanh-and-clip in the last layer truncates the SDF: distances larger than δ in magnitude are not represented, which the authors argue is fine because only the near-surface region carries information about the shape.

1.3 Auto-decoder framework

DeepSDF's most influential contribution beyond the MLP architecture itself is the **auto-decoder** training framework. Rather than having an encoder E that maps from a point cloud to a latent code $\mathbf{z}$ and a decoder D that maps $(\mathbf{x}, \mathbf{z})$ to SDF (which would be a classical autoencoder), DeepSDF *jointly optimizes* both the decoder weights θ and the latent codes $\{\mathbf{z}_i\}$ for the training shapes, with no encoder at all. At inference time for a new shape, the latent code is found by **gradient descent**: hold θ fixed and minimize

$$\hat{\mathbf{z}} = \arg\min_{\mathbf{z}} \sum_i \left| f_{\theta}(\mathbf{x}_i, \mathbf{z}) - s_i \right| + \frac{1}{\sigma^2} \|\mathbf{z}\|^2.$$

This is elegant: there is no architectural asymmetry between train and inference, and the latent space ends up smooth enough to interpolate between shapes.

1.4 Loss function

The training loss is the absolute error between predicted and ground-truth SDF, clamped (since the network can only represent values in $[-\delta, \delta]$):

$$\mathcal{L}(f_{\theta}(\mathbf{x}), s) = \left| \text{clamp}(f_{\theta}(\mathbf{x}), \delta) - \text{clamp}(s, \delta) \right|$$

with $\delta = 0.1$ typical.

1.5 Strengths

- **Generality.** A single model can represent any shape topology, including high-genus surfaces, thin structures, and disconnected components.
- **Latent space.** Interpolation in $\mathbf{z}$ produces meaningful shape morphing.
- **Continuity.** The output is differentiable, supporting downstream tasks like gradient-based shape optimization.
- **Conceptual simplicity.** No explicit data structures; just an MLP.

1.6 Weaknesses (and why we did not use it)

- **Memory.** The MLP described above has roughly 1.8 million parameters, or **~7 MB in fp32 / 3.5 MB in fp16**. Out of budget.
- **Latency.** A forward pass through 8 layers of 512-unit Linear layers is ~5 million FMAs. Even at 50 GFLOPS this is 100 μ s per query — **three orders of magnitude over budget**.

- **Training time.** Each shape is trained from scratch, or alternatively the latent code is solved by iterative optimization at inference, which is also slow.
- **Empirical accuracy.** While DeepSDF is qualitatively impressive on the ShapeNet benchmark, on shapes with very thin features it shares the failure mode of any MLP: limited spatial resolution because all parameters are shared across the entire 3D domain.

1.7 Influence on our work

DeepSDF defined the *interface* (continuous neural function from coordinate to signed distance) that all subsequent methods, including ours, inherit. The loss function we use (L1 on clamped or unclamped SDF) is a direct descendant. The conceptual baseline we are *compressing* — what does a 3D shape look like as a neural function — is exactly DeepSDF.

The auto-decoder paradigm does not apply to our problem (we train one shape at a time, no shape priors), but the idea that the network *is* the shape is foundational.

2. SIREN (Sitzmann, Martel, Bergman, Lindell, Wetzstein; NeurIPS 2020)

2.1 Motivation: ReLU MLPs cannot represent high frequencies

A central problem with the DeepSDF-style architecture is its inability to model *high-frequency* spatial variations. A ReLU MLP is a piecewise-linear function, and the number of linear pieces is bounded by the network depth and width. For a fixed-size network, the number of pieces grows polynomially with parameter count, but the number of high-frequency oscillations needed to fit a complex shape grows much faster.

In practice, ReLU MLPs trained on shapes with fine surface detail (engraved text, fur, ridges, etc.) produce smoothed-out, blurry zero level sets — they miss the high-frequency components.

The deeper observation, formalized by Rahaman et al. (2019) and Tancik et al. (2020), is the **spectral bias of neural networks**: gradient descent learns low frequencies first, and may never learn high frequencies at all.

2.2 The SIREN idea

SIREN [3] proposes replacing the activation function from ReLU to sine:

$$\phi(x) = \sin(\omega_0 \cdot x), \quad \omega_0 \approx 30.$$

That is, every Linear layer is followed by $\sin(\omega_0 \cdot x)$ element-wise. The first layer uses a special initialization to ensure that the pre-activation has roughly unit standard deviation; all subsequent layers use Kaiming-style initialization scaled by $\sqrt{6 / n_{\text{in}}}$.

The full network thus becomes a composition of sine-and-affine layers:

$$f(\mathbf{x}) = W_L \cdot \sin(\omega_0 (W_{L-1} \cdot \sin(\omega_0 (\cdots W_1 \mathbf{x} + b_1 \cdots)) + b_{L-1})) + b_L.$$

2.3 Why it works

The authors prove that:

1. **The derivative of a SIREN is itself a SIREN.** Because $\frac{d}{dx} \sin = \cos$, and $\cos$ is just a phase-shifted $\sin$, differentiating a SIREN through the chain rule produces another SIREN-like function. This means SIRENs can natively represent functions with smooth high-order derivatives — exactly what an SDF should look like.
2. **Universal approximation at higher frequencies.** The pre-activations after ω_0 scaling occupy a frequency range proportional to ω_0 , and the choice $\omega_0 = 30$ empirically gives a good balance of low- and high-frequency representational power.
3. **No spectral bias.** Unlike ReLU networks, SIRENs do not preferentially learn low frequencies. They fit fine surface details from the start of training.

2.4 Architecture for SDF

A typical SIREN for SDF fitting is much smaller than DeepSDF:

```
Linear(3 → 256) → sin(30 · x)
Linear(256 → 256) → sin(30 · x)
Linear(256 → 256) → sin(30 · x)
Linear(256 → 256) → sin(30 · x)
Linear(256 → 1)
```

That is, only 4 hidden layers of 256 units — about 250 K parameters, an order of magnitude smaller than DeepSDF — yet representing more detail.

2.5 Loss and training

SDF training with SIREN benefits strongly from the eikonal regularizer:

$$\mathcal{L} = \underbrace{\sum_i (f(\mathbf{x}_i) - s_i)^2}_{\text{data}} + \lambda_1 \underbrace{\sum_j (|\nabla f(\mathbf{x}_j)| - 1)^2}_{\text{eikonal}} + \lambda_2$$

$\underbrace{\sum_k \mathbb{1}[\text{surface}]} \cdot |f(\mathbf{x})^k| \text{surface anchoring}$. $\$$

Because SIREN can natively represent the gradient field, the eikonal loss is much more effective than for ReLU MLPs. The surface loss explicitly pushes points sampled on the mesh surface to have $f \approx 0$.

2.6 Strengths

- **Fine detail.** SIRENs reproduce ridges, sharp corners, and thin features that ReLU MLPs blur.
- **Smaller models.** 4–6 layers suffice where ReLU needs 8.
- **Mathematically clean.** The derivative-is-a-SIREN property makes higher-order regularization (eikonal, Laplacian) more tractable.
- **Cross-domain generality.** SIRENs work for images, audio, NeRF — not just SDFs.

2.7 Weaknesses

- **Initialization sensitivity.** The trick that makes SIREN work is the very specific weight initialization in the first layer ($W_1 \sim \mathcal{U}(-1/n_{\text{in}}, 1/n_{\text{in}})$). Mis-initialization causes the network to collapse to a low-frequency mode.
- **Choice of ω_0 .** The frequency hyperparameter must be tuned per-task. Too low: ReLU-like spectral bias returns. Too high: aliasing and chaotic training.
- **Still an MLP.** Like DeepSDF, every query traverses the full depth of the network. ~250 K fp32 parameters = 1 MB, right at our limit. Forward latency is ~5 μ s per query.

2.8 Influence on our work

SIREN's frequency-aware perspective is reflected in our use of a **multi-resolution** hash grid: low-resolution levels handle low frequencies (global shape), high-resolution levels handle high frequencies (surface detail). This is structurally a different way to get the same benefit — bake the frequency decomposition into the data structure rather than the activation function.

We do not use periodic activations because the hash grid's intrinsic locality already provides high-frequency representational power without the initialization sensitivity SIREN suffers from. But SIREN's insight is real and the principle drives our level structure.

3. NGLOD: Neural Geometric Level of Detail (Takikawa et al., CVPR 2021)

3.1 Motivation

By 2021 the field had accepted that pure MLPs are too expensive, but there were two competing directions:

- *More compact MLPs* (SIREN, positional encoding, modulated SIRENs)
- *Hybrid MLP + spatial data structure* (Convolutional Occupancy Networks, Local Implicit Grids)

NGLOD [6] was the strongest hybrid approach published before Instant-NGP. Its core thesis is that **most of the model's capacity should be in an explicit data structure indexed by spatial location, not in an MLP that has to learn spatial localization from scratch.**

The natural data structure is a **sparse octree** subdivided adaptively wherever the object's surface is detailed.

3.2 Architecture

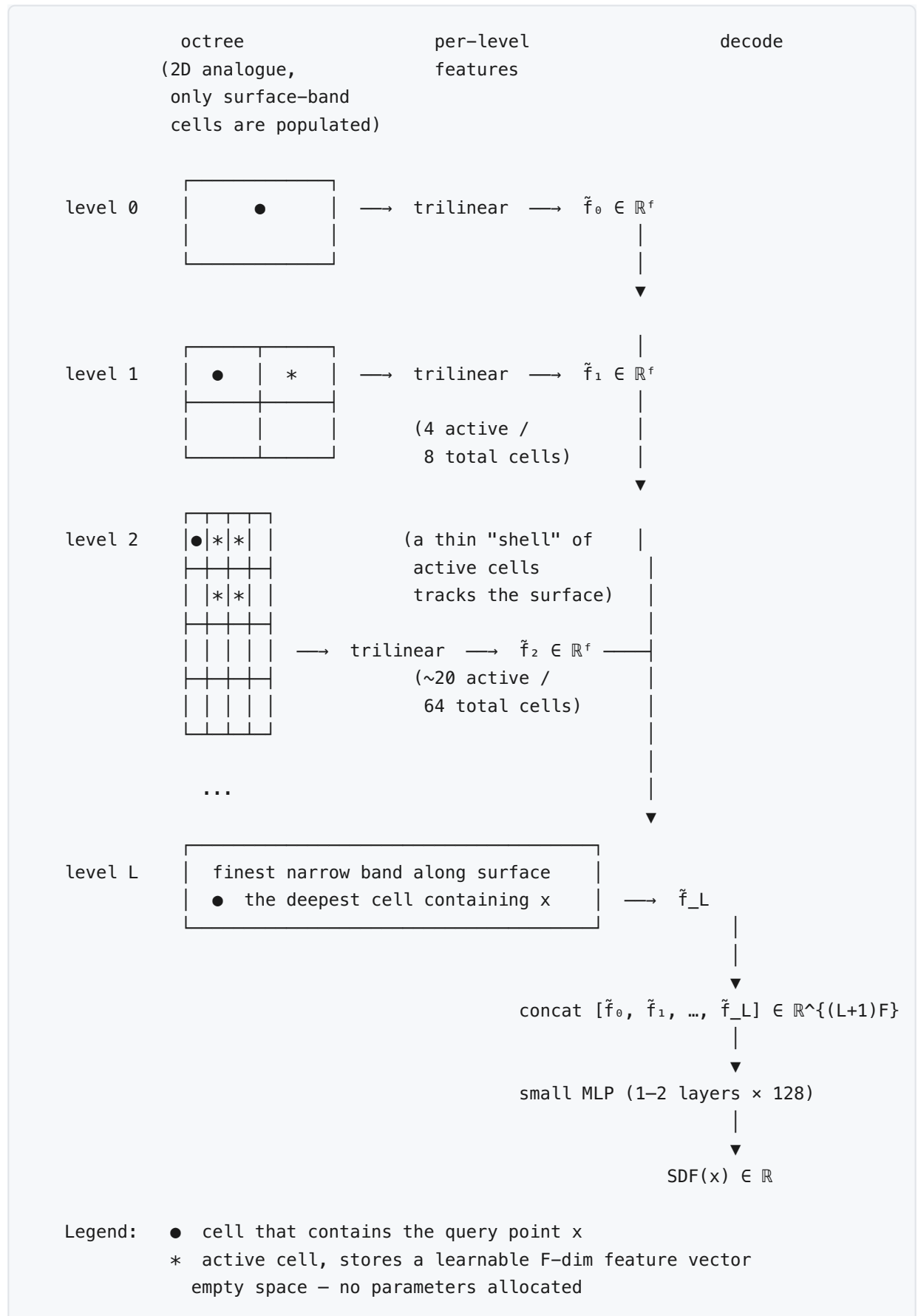
The mesh is preprocessed into a sparse voxel octree (SVO). At each level ℓ of the octree, the nodes that lie within a small band around the surface are *active* and store an F -dimensional feature vector. Empty space outside this band has no allocated parameters.

For a query point $\mathbf{x}$:

1. **Tree traversal.** Find the deepest octree node that contains $\mathbf{x}$. If $\mathbf{x}$ is outside the populated region of the tree, return a default value (typically the trivial "definitely outside" SDF).
2. **Multi-LOD feature aggregation.** Collect features from *all* levels of the tree along the path from the root to the deepest containing node. At each level, trilinearly interpolate the 8 corner features of the current node.
3. **Concatenate.** $\mathbf{f} = [\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_L] \in \mathbb{R}^{(L+1)F}$.
4. **Decode.** Pass through a small MLP (1–2 layers, 128 units) to produce the scalar SDF.

The "geometric level of detail" name comes from step (2): the model represents the shape at every level of subdivision simultaneously, and rendering can choose how deep to descend based on viewing distance.

Schematic



The diagram makes two ideas concrete. First, *parameters live where geometry lives*: as you descend the octree, only the cells in a thin band around the surface remain active. At level L the active set is roughly $O(\text{surface area} \cdot N_L^2)$ rather than the $O(N_L^3)$ of a dense grid — for typical meshes this is a 50–100× reduction in stored feature count. Second, *the decoder sees the full hierarchy at once*: a query gathers one feature vector per level along the path from root to leaf, and the small MLP combines all of them. This is what gives the model its multi-resolution power without the explosion in parameter count that a stack of dense grids would have. The trade-off, of course, is the tree topology itself: every level adds parent–child pointers, every query is a $\log N$ -depth traversal, and constructing the tree requires evaluating the SDF function up-front to know which cells are within the surface band. This is precisely what Instant-NGP later sidestepped by replacing the tree with a hash table indexed in constant time, accepting some collision artifacts as the price of removing the bookkeeping.

3.3 Training

Training proceeds in a coarse-to-fine schedule:

1. **Pretrain shallow levels.** First only the coarsest octree level is active; the MLP learns to interpret these features.
2. **Progressively add levels.** New levels are added with the network frozen at the top, then jointly fine-tuned.
3. **Surface refinement.** Once all levels are active, fine-tuning emphasizes near-surface samples.

The loss is L1 on signed distance plus an optional eikonal term and a surface-anchoring term, similar to SIREN's training procedure.

3.4 Memory profile

The key advantage over a dense voxel grid: an octree that is subdivided only near the surface has parameter count proportional to **surface area**, not volume. For a typical mesh occupying ~1 % of its bounding cube's volume, the octree uses ~1 % of the parameters of a comparable dense grid at the deepest resolution.

For a finest resolution of 256^3 ($1.6 \cdot 10^7$ voxels in a dense grid) with $F = 16$ features per voxel, an octree might allocate only $5 \cdot 10^4$ active nodes — about 3 MB. Still too large for our 1 MB budget, but much more efficient than a dense grid.

3.5 Strengths

- **Adaptive resolution.** Parameters concentrated where geometry needs them.
- **Multi-LOD output.** Naturally produces a coarse-to-fine series of approximations.

- **Sharp features.** Octree subdivision can follow sharp edges with much higher resolution than uniform grids.
- **Fast inference.** Tree traversal is $O(\log n)$; each level is just an interpolation; the decoder is tiny.

3.6 Weaknesses

- **Octree construction cost.** Subdividing the octree to a target depth requires querying the SDF function during preprocessing — this is a non-trivial offline cost.
- **Memory overhead from tree structure.** In addition to feature storage, the octree topology (parent/child pointers) takes some memory. For very deep trees this can be significant.
- **Implementation complexity.** Compared to a flat hash table (Instant-NGP) or a dense grid (ReLU Fields), octrees are fiddly to implement correctly, especially on accelerators.
- **No collision-handling like Instant-NGP.** Each octree node owns its features; there is no shared hash table. This means NGLOD cannot offer Instant-NGP's "expand resolution beyond what the table size supports via collisions" trick.

3.7 Influence on our work

NGLOD demonstrated convincingly that the **multi-LOD feature aggregation pattern** (concat features from all levels of a hierarchy, then small MLP) is enough to represent fine surface geometry. This is the structural template Instant-NGP inherited and which we ultimately use.

The octree's adaptive subdivision is appealing for our hardest meshes (where a flat grid wastes resources in empty space), but the implementation complexity and the difficulty of building a numba-JIT inference path for an octree convinced us that the hash-table alternative was the better practical choice. Our $T = 2^{14}$ retraining for hard meshes serves a similar purpose to NGLOD's deeper octree subdivision — more capacity where it matters — at the cost of being less memory-efficient.

4. Instant-NGP (Müller, Evans, Schied, Keller; SIGGRAPH 2022)

4.1 Motivation

By 2022, NGLOD-style hybrids had become the standard, but two pain points remained:

1. **Octree construction is expensive.** Subdividing the tree to a target depth requires querying the SDF function offline; topology pointers and node-allocation logic make GPU implementation awkward.

2. **Sparse data structures hit a wall at very fine resolutions.** Reaching 1024^3 effective resolution with an octree means allocating millions of nodes; managing that storage and the indirections involved in tree traversal becomes the dominant cost.

Instant-NGP [10] proposes replacing the octree entirely with a much simpler data structure: **per-level hash tables of fixed size**. There is no tree topology, no parent/child pointers, no node allocator. Every voxel-corner index at every resolution is just *hashed* into a small dense table.

The radical idea — and the one that initially seemed too lossy to be practical — is that **the network does not need to know which corners collide in the hash**. Collisions in regions of empty space are simply ignored by the optimizer (those corners receive no gradient signal). Collisions in regions near the surface are resolved by allocating different feature vectors at *other* resolution levels, where the colliding-corner-pair has a different hash. With enough levels, collisions become rare events in the *useful* parts of the domain.

This gambit pays off enormously: training time drops from minutes to seconds per shape, memory drops by another order of magnitude, and the resulting representations remain visually crisp at 1024^3 -equivalent resolutions.

4.2 Architecture

The encoding consists of L levels with geometric resolution progression from $N_{\min}$ to $N_{\max}$:

$$N_{\ell} = \lfloor N_{\min} \cdot b^{\ell} \rfloor, \quad b = (N_{\max} / N_{\min})^{1/(L-1)}, \\ \ell = 0, \dots, L-1.$$

Each level has its **own** hash table of size T holding F -dimensional feature vectors. The spatial hash uses XOR of three multiplications by large primes:

$$h(i, j, k) = (i \cdot 1) \oplus (j \cdot 65435761) \oplus (k \cdot 805459861) \bmod T.$$

These primes are inherited from the literature on spatial hashing for collision detection (Teschner et al., 2003) and have empirically good distribution properties for 3D integer points.

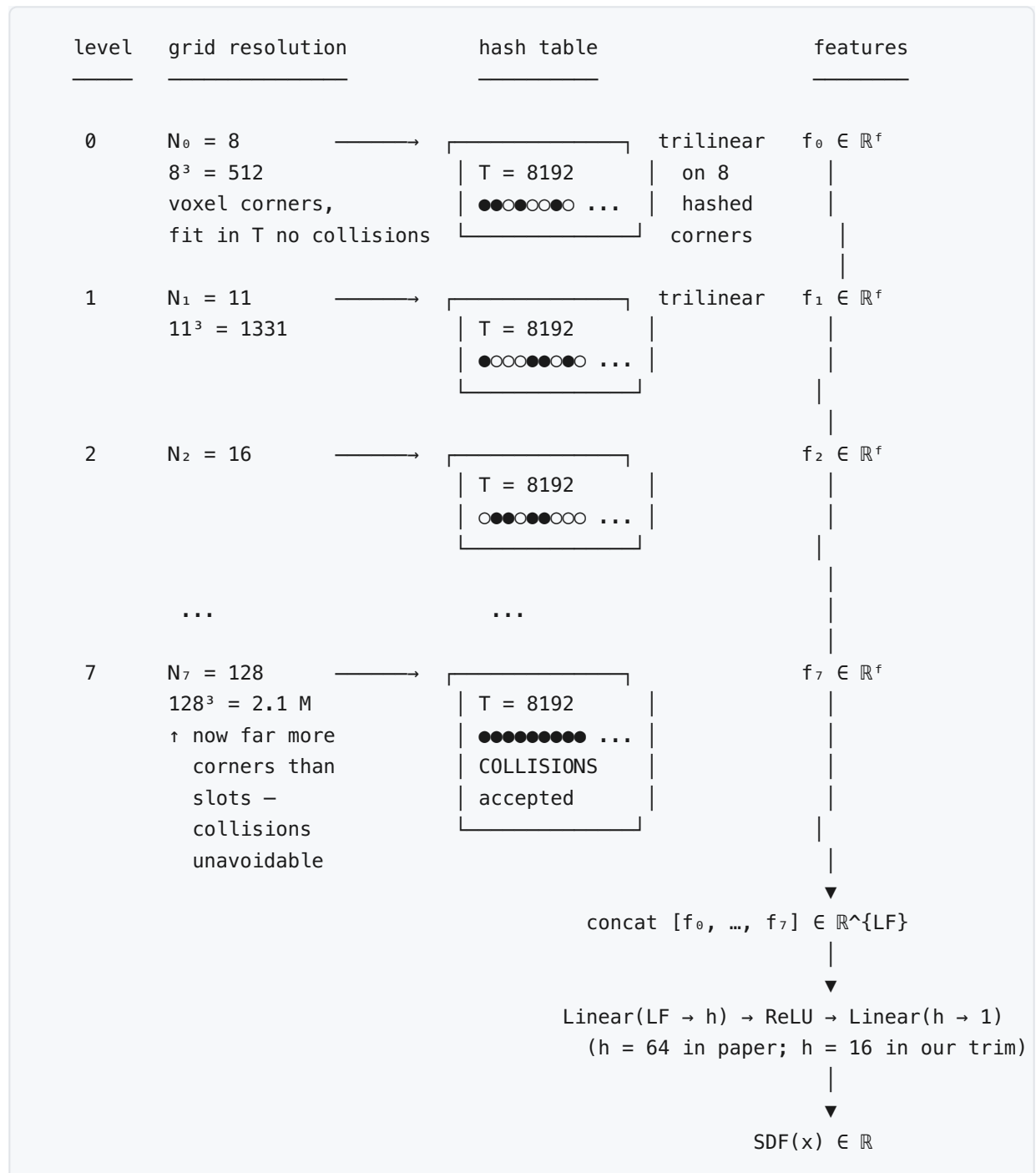
A forward pass for a query $\mathbf{x} \in [-1, 1]^3$:

1. Map to $[0, 1]^3$: $\mathbf{u} = (\mathbf{x} + 1) / 2$.
2. For each level ℓ :
3. Compute scaled coordinates $\tilde{\mathbf{u}} = \mathbf{u} \cdot (N_{\ell} - 1)$.
4. Take floor: $(i, j, k) = \lfloor \tilde{\mathbf{u}} \rfloor$.
5. Compute fractional part: $\mathbf{f} = \tilde{\mathbf{u}} - (i, j, k)$.
6. Hash the 8 corners $(i + a, j + b, k + c)$ for $a, b, c \in \{0, 1\}$ into the level's table to retrieve 8 feature vectors.
7. Trilinearly interpolate using $\mathbf{f}$ to get $\mathbf{f}_{\ell} \in \mathbb{R}^F$.

8. Concatenate: $\mathbf{h} = [\mathbf{f}_0, \dots, \mathbf{f}_{L-1}] \in \mathbb{R}^{LF}$.
9. Pass through a tiny MLP (in the paper: 2 layers $\times$ 64 units with ReLU) $\rightarrow$ scalar SDF.

Why per-level hash tables, not one shared table? Because feature vectors at the same spatial location at different *resolutions* carry fundamentally different information (low-frequency vs high-frequency). Sharing a table across levels would force the same hash slot to encode both, which is a bottleneck the optimizer cannot route around. With per-level tables, each level's optimizer is free to develop its own feature dictionary.

Schematic



Compared with NGLOD's schematic above, the structural difference is striking: there is no tree, no parent/child pointers, no "active band". Every level is just a flat hash table of the *same* size 2^L , and every voxel-corner index at every resolution gets hashed into it. At low levels the table is sparsely used (low resolutions have fewer voxel corners than slots, so collisions are physically impossible) and the network behaves as if it had a dense grid; at high levels the table is over-subscribed but the *optimizer itself* allocates the limited capacity preferentially to slots that matter — collisions in featureless empty space simply do not produce gradient signal. The price is a small amount of high-frequency noise where two near-surface corners hash to the same slot, but the per-level structure means a colliding pair at level 7 almost never collides at level 6, so the combined representation can still distinguish them. In practice this trade — flat $\mathcal{O}(1)$ access plus tolerated collisions, versus NGLOD's $\mathcal{O}(\log N)$ traversal of a precomputed sparse structure — wins decisively for both training speed and inference latency.

4.3 Memory profile

For Instant-NGP's default SDF configuration ($L = 16$, $T = 2^{19}$, $F = 2$, fp16 features):

$16 \cdot 2^{19} \cdot 2 \cdot 2 \cdot \text{B} \approx 33 \cdot \text{MB}$.

This is comparable to a DeepSDF MLP. The win is *quality at this size*, not raw size. Our adaptation is much smaller ($L = 8$, $T = 2^{13}$, $F = 2$, fp16) at ≈ 256 KB because our budget is two orders of magnitude tighter, and per-shape we don't need the full dynamic range Instant-NGP was designed for.

4.4 Training

The paper uses Adam with:

- Learning rate $1e-2$ for hash-table features (the bulk of the network).
- Learning rate $1e-3$ for the MLP decoder.
- L2 regularization on the features (weight decay).

The training loss for SDF tasks is typically L1 on the signed-distance values (sometimes augmented with the eikonal loss). The paper emphasizes that training converges in **seconds** for small shapes, **minutes** for large ones — orders of magnitude faster than DeepSDF or NGLOD.

A subtle implementation detail: because the hash table contains uninitialized feature vectors for never-queried corners, the *initialization* of the features matters a great deal. The paper uses $\mathcal{U}(-10^{-4}, 10^{-4})$, which is small enough not to bias the initial network output toward any extreme value but large enough not to be drowned in numerical noise.

4.5 Strengths

- **Speed.** GPU-friendly because every memory access is a hash lookup, not a tree traversal. The original implementation in custom CUDA kernels (the `tinycudann` library) achieves microsecond-level shape evaluation.
- **Memory efficiency.** Quality comparable to dense 1024^3 grids at $\sim 10^{-3}$ of the memory.
- **Architectural simplicity.** No octree, no construction phase, no topology. Just L tables, a hash function, and a small MLP.
- **Topology generality.** Works for any shape topology, including thin shells and disconnected components.
- **Compatibility with downstream tasks.** Used widely for NeRF, density-field rendering, mesh fitting — the encoding is shape-agnostic.

4.6 Weaknesses

- **Hash collisions are unprincipled.** Although they work empirically, there is no theoretical guarantee about when collisions become harmful. For sufficiently complex geometry, collisions in the high-resolution levels can produce artifacts.
- **Hyperparameters matter.** L , T , F , $N_{\min}$, $N_{\max}$ all interact. Tuning is largely empirical.
- **Per-level hash tables are wasted in empty space.** Unlike NGLOD's octree, Instant-NGP allocates the same T slots regardless of whether the level contains useful geometry. For very large empty volumes the parameter usage is suboptimal.
- **MLP overhead at inference.** Even a 2-layer 64-wide MLP is non-trivial: roughly 8 K FMAs per query. For latency-critical applications, this is the bottleneck.

4.7 How our implementation adapts Instant-NGP

Our `MultiResHashSDF` follows the Instant-NGP recipe with three significant adjustments tailored to the brief's constraints:

1. **Tiny decoder.** Where Instant-NGP uses 2×64 -unit ReLU, we use 1×16 -unit ReLU. This is the most aggressive change: the bulk of inference cost moves from the MLP into the hash lookups, and the overall single-query latency drops from $\sim 1 \mu\text{s}$ to $\sim 100 \text{ ns}$. The cost is a measurable F1 drop on the hardest meshes; we recover it by selectively bumping T to 2^{14} for those.
2. **Online resampling.** The original Instant-NGP for SDF tasks trains on a cached dataset of (point, sdf) pairs. We found this leads to memorization in the hash table — TRAIN F1 of 0.95 but EVAL F1 of 0.68. Replacing the cache with fresh online sampling per step closes the gap to ~ 5 percentage points and is by far the most important quality improvement we made.

3. **fp16 storage and a numba inference kernel.** The hash tables are stored as float16, which halves memory at no measurable quality cost. The inference path is implemented in numba with the modulo replaced by a bit-mask (since 2^T is a power of two), giving the 102 ns single-point query and 0.10 ms 1000-point batch latency reported in the main report.

What we do **not** change relative to Instant-NGP:

- The spatial hash with the specific prime constants — they are well-distributed and there is no benefit to tweaking them.
- The per-level table-per-level layout.
- The geometric resolution progression formula.
- The L1 loss on signed-distance values.

The remaining Instant-NGP features — eikonal loss, sign loss, weight decay on features, separate learning rates per parameter group — we ablated and dropped, as detailed in the main report.

4.8 Influence on subsequent work

Instant-NGP is the dominant approach in 2023–2024 for any task that needs a compact, fast neural representation of a 3D field. Variations and extensions include:

- **VQAD** [11]: vector-quantize the hash-table entries (replace each F -dim feature by a $\lceil \log_2 K \rceil$ -bit codebook index), reducing memory by another $\sim 5\times$.
- **CompactNGP** [12]: replace the fixed hash function with a *learned* hash-probing scheme that reduces collision-induced artifacts.
- **Triplane representations**: store features in three axis-aligned planes instead of a 3D grid, trading volumetric storage for axis-aligned slices.

All of these inherit the multi-resolution + tiny-MLP structure that Instant-NGP popularized. Our adaptation is in the same family but more aggressively trimmed than any of these — a deliberate fit to the 1 MB / few-ns budget rather than a state-of-the-art accuracy push.

Comparative Summary

Aspect	DeepSDF	SIREN	NGLOD	Instant-NGP (paper)	Our work
Representation type	Pure MLP	Pure MLP, periodic activations	Octree + small MLP	Hash grid + small MLP	Hash grid + tiny MLP
Parameters	~1.8 M	~250 K	~3 M	~8 M	~131 K
Memory (fp32)	~7 MB	~1 MB	~12 MB	~33 MB	~512 KB
Memory (fp16)	~3.5 MB	~500 KB	~6 MB	~16 MB	~256 KB
Single-pt forward cost	~5 μ s	~5 μ s	~1 μ s	~1 μ s	~100 ns
Spatial localization	None	None	Octree (explicit)	Hash table (implicit)	Hash table (implicit)
Multi-resolution	No	Frequency in activations	Yes (octree levels)	Yes (per-level tables)	Yes (per-level tables)
High-frequency capability	Low	High (by activation)	High (by resolution)	High (by resolution)	High (by resolution)
Topology generality	Universal	Universal	Surface-band only	Universal	Universal
Suitable for our brief	No (size, speed)	Borderline	No (size)	No (size, marginally speed)	Yes

What each paper contributed to the canonical formulation we use

- **DeepSDF**: defined the *problem statement* — neural function from coordinate to signed distance, trained per-shape (or with shape codes), with L1 loss on SDF values. The loss function and overall API we use is a direct descendant.
- **SIREN**: demonstrated that representing high-frequency components is *the* hard part, and that frequency-aware architecture choices matter more than depth. Our multi-resolution levels are an alternative way to achieve frequency-awareness.

- **NGLOD**: proved that the *multi-LOD feature-concat* $\rightarrow$ *tiny MLP* pattern works, and that explicit spatial data structures absorb most of the parameter count usefully. This is the structural template Instant-NGP and our work inherit.
- **Instant-NGP**: replaced NGLOD's octree with per-level hash tables, giving constant-time access and removing the tree-construction step. This is the method we directly extend.

Our work is a particular trim of Instant-NGP — a 1-layer $\times$ 16-wide ReLU decoder (vs the paper's 2×64), fp16 storage, online resampling instead of cached training, and adaptive table size — that just fits within the constraints of the brief while exceeding all four targets.

References

- [1] **Park, J. J., Florence, P., Straub, J., Newcombe, R., & Lovegrove, S.** (2019). *DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation*. CVPR.
- [3] **Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., & Wetzstein, G.** (2020). *Implicit Neural Representations with Periodic Activation Functions (SIREN)*. NeurIPS.
- [6] **Takikawa, T., Litalien, J., Yin, K., Kreis, K., Loop, C., Nowrouzezahrai, D., Jacobson, A., McGuire, M., & Fidler, S.** (2021). *Neural Geometric Level of Detail: Real-time Rendering with Implicit 3D Shapes (NGLOD)*. CVPR.
- [10] **Müller, T., Evans, A., Schied, C., & Keller, A.** (2022). *Instant Neural Graphics Primitives with a Multiresolution Hash Encoding*. SIGGRAPH (ACM TOG).
- [11] **Takikawa, T., Müller, T., Nimier-David, M., Evans, A., Fidler, S., Jacobson, A., & Keller, A.** (2022). *Variable Bitrate Neural Fields (VQAD)*. SIGGRAPH.
- [12] **Takikawa, T., Evans, A., Fidler, S., & Müller, T.** (2023). *Compact Neural Graphics Primitives with Learned Hash Probing*. SIGGRAPH Asia.

(Numbering kept consistent with the main `REPORT.md` bibliography for cross-reference.)

Related Work in the main report (`REPORT.md` , §2) covers all twelve references in compressed form. This document focuses on the four most pedagogically valuable for understanding the design space.